Lappeenrannan teknillinen yliopisto
School of Business and Management

Software Development Skills: Full-Stack

Diako Jalal, 2524490

LEARNING DIARY, FULL-STACK MODULE

LEARNING DIARY

22.6.-24.6.2026 - Getting set up and getting oriented

Initially I scrolled through the overview of the course then I decided on the full-stack module. Then I began customizing my environment: installing Node (checking it is working with node -v) and installing VS Code with the addons I thought I could use, and I added the MongoDB and REST Client extensions a few days later once I knew what to do with them. I took more time than I should have on selecting addons as I read too many 'best extensions' lists.
What I didn't do was put anything under version control, and that's what I kind of regret. I wanted to start writing code, a plain folder was fine and setting up a repository felt like admin I could do when I had something worth committing. This diary entry is what that cost me.
I also looked through the first part of the MERN example project playlist to get an idea of how the four technologies form a bigger whole, before I knew any of them in detail. It didn't leave much detailed information in my head but it left me with a diagram-React on the frontend sending requests over HTTP to Express on the server, which then interacts with a Mongo database using Mongoose. That diagram helped the four topics settle into my mind as pieces of a unified structure rather than four separate areas.

I learned about,

- what the four technologies are for and roughly where each one sits, before learning any of them properly.
- how to set up VS Code for this stack, and that picking tools is not the same as making progress.
- version control in principle, from the intro video.
- and, in hindsight, that understanding version control in principle is not the same as using it. See 25.7.-30.7.

25.6.-29.6.2026 - Node.js core

I worked through the Node intro and crash course, typing everything rather than watching. Then I closed the video and rebuilt the pieces from a blank editor, which is where I found out what I had actually retained: a module that exports a function and is required from another file, reading a file with fs.readFile callbacks and then again with fs/promises and async/await, a class, a class that extends EventEmitter, and finally an HTTP server using the http module only.

Mistakes. I logged a value and got Promise { <pending> } instead of my data, twice, before the pattern registered - that print always means a missing await. I then moved the await to the top level of the file and got a syntax error, and had to wrap it in const main = async () => {...} with a main() call. I also lost time to "undefined is not a function", which turned out to be module.exports = thing being imported as const { thing } = require(...). The two export forms need matching import forms and nothing warns you.

The most useful part was the bare HTTP server. Hand-writing the routing - checking req.url and req.method myself for every route, setting Content-Type myself - is tedious in a way that is hard to appreciate from a description. As an experiment I added a route returning HTML instead of JSON, which meant setting the header by hand again. That tedium is the entire argument for Express, and I would rather have felt it than been told it.

I also put console.log on the lines before and after an async call and watched the order come out differently from how I read the file. Code stops running top to bottom the moment callbacks are involved.

I learned about,

- Promise { <pending> } as a diagnostic rather than an error.
- the difference between module.exports = thing and module.exports = { thing }, and the import each one needs.

- EventEmitter as one of the patterns Node is built out of - it reappears in streams and inside http - not a side feature.
- why Express exists, from the inside.

30.6.-4.7.2026 - MongoDB and Mongoose

I installed MongoDB with the "install as a service" option, plus Compass. Before touching Mongoose I inserted, queried, updated and deleted documents by hand in the shell and in Compass, which I am glad I did: being able to see the documents turned database work from guesswork into something I could observe. Then I defined schemas in Mongoose and did the same CRUD from a .js file, added a second collection with a ref, loaded both together with populate, and wrote one aggregate pipeline with $group.

Two failures worth recording. The first was silent - nothing connected and there was no error at all. The mongod service was not running. sc query MongoDB and looking for STATE : 4 RUNNING is now the first thing I check, and mongod --dbpath <folder> works as a manual fallback. The second cost me over an hour: I compared two ObjectIds with === and got false for two ids that were plainly identical in Compass. They are objects, not strings, so === compares references; .toString() on both sides fixed it. Earlier I had also forgotten populate and spent a while wondering why a field was a meaningless string of hex. That string was the ObjectId, which is exactly what you get without populate.

I read that Mongoose validation runs in my Node process and not in the database, so it does not protect the data from anything writing directly to MongoDB, and that findByIdAndUpdate does not run validators unless you pass { runValidators: true }. That one lets bad data in without complaining, which is worse than an error.

Experiment: I left Compass open while the app was running, added a record in the browser and refreshed Compass to watch the document appear, then edited it in Compass and refreshed the browser. Seeing it work in both directions is what made "the database is a separate program" concrete instead of a sentence I could repeat.

I learned about,

- populate being a second query that Mongoose runs for me rather than magic, and $group doing the grouping inside the database instead.
- the tradeoff between them as a real decision: my stats endpoint ends up counting in plain JavaScript, because at this data size following and debugging the logic mattered more than the speed, and I would rather explain that choice than have made the clever one.
- ObjectId comparison, and that === on objects is about references.
- validation living in my process, and { runValidators: true }.

5.7.-9.7.2026 - Express

I coded along with the Express crash course and its source repository, stopping at each app.use to say out loud what it did. I rebuilt the Module 1 HTTP server in Express and the difference in length answered my own complaint from the week before. Then I split routes into a Router in its own file, wrote two pieces of my own middleware - one logging the method and URL on every request, one validating a request body and returning 400 - and added 404 and error handlers at the bottom.

At the bottom, eventually. My first version returned 404 for every single request, including ones I could see registered, because the 404 handler was above the routes. Middleware runs in the order you register it, and once that clicked, app.use stopped feeling arbitrary and started feeling like the whole model. I also hit req.body being undefined (missing app.use(express.json())) and "Cannot set headers after they are sent" from responding twice in one handler, which is why I now write return res.json(...) by habit. One more: /courses/new was being swallowed by /courses/:id because :id was registered first, with :id equal to the string "new".

I tested every route with the REST Client extension and a requests.http file rather than the browser, because a browser can only send GET and I would otherwise have had to build a UI before I could test a POST.

The best thing that happened this week was finding a bug in the teaching material. The tutorial error handler contains const statusCode = res.statusCode ? res.statusCode : 500. res.statusCode starts at 200 and 200 is truthy, so that expression can never pick 500 - every unhandled error comes back as "200 OK" with an error message in the body. I only noticed because I deliberately threw an error in a route to watch the handler run, then set res.status(400) before the throw and watched the status change. I also read that in plain Express a rejected promise inside an async handler never reaches the error handler at all; it just vanishes. That is what express-async-handler solves, which explains why it turns up in the MERN module.

I learned about,

- middleware order as the model, not a detail.
- res.status(400) before throw as the mechanism behind the error pattern used in the MERN module.
- what express-async-handler saves me from writing.
- the routes, then controllers, then middleware layering - the same shape my own backend uses, with Mongoose instead of an array in memory.

10.7.-14.7.2026 - React fundamentals

I started the React crash course and read react.dev/learn alongside it, which was worth the extra time. I built a counter with useState, then a task list, then moved the task state up into the parent and passed handlers down through props. That last step is lifting state up and it took a while to click - the version where state lives in the child looks fine right up until two components need the same data.

The mistake that taught me most: tasks.push(newTask) followed by setTasks(tasks) did nothing at all on screen. No error, no warning, just a UI that would not change. React compares by reference and I had handed it back the same array. setTasks([...tasks, newTask]) works because it is a new array. I now ask "did I create a new object, or mutate the old one?" first whenever something does not update, and it has already saved me time twice.

I also wrote onClick={handleClick()} instead of onClick={handleClick} and got an immediate re-render loop, because the first form calls the function during render and mine set state. And I expected setCount(count + 1) followed by console.log(count) to show the new value; state updates are queued for the next render, so it shows the old one.

Then a controlled form, where every input value comes from state. Once I had written one the pattern was obvious, and it is the same pattern in every form in my own project.

I learned about,

- immutability as the mechanism behind rendering, not a style preference.
- lifting state up, and why threading handlers through props gets ugly once several components need the same data. That is the argument for Redux in the next module, so I deliberately did not reach for it here.
- passing a function reference versus calling it in JSX.
- state updates being queued rather than immediate.

15.7.-18.7.2026 - Effects, data fetching and routing

I carried on with useEffect, fetching from a public API and rendering the result, then added React Router with two pages and a link between them.

I produced my first infinite re-render loop: a useEffect with no dependency array that called setState. Render triggers the effect, the effect triggers a render, and the laptop fan tells you before the console does. The dependency array is not optional. I also had a list that showed the wrong row after I deleted an item from the middle, which was an index-based key; switching to the database _id fixed it and explained what key is actually for. And session.course.code crashed on the first render because the data had not arrived yet - guarding with session?.course?.code or an early return is automatic for me now.

I added loading and error states to every fetch. The course says this is not optional and I agree after this week: a real request has three outcomes, not one, and a component written only for the success case looks fine locally and breaks the first time the server is slow.

One thing I noticed rather than was told: the GitHub search in the exercise fires a request on every keystroke. That is fine for an exercise, but in a real app I would delay it until typing stops and cancel the in-flight request when a new one starts, or responses can arrive out of order and the list ends up showing answers to a query I have already replaced. I did not implement it here, but I know what it would involve.

I learned about,

- the dependency array as the thing that stops an effect re-running forever.
- key needing to be stable identity, not position.
- optional chaining as first-render defence.
- loading and error states as the normal case, and debouncing plus cancellation as the real-world version of fetch-on-type.

19.7.-24.7.2026 - MERN together, and authentication

Here is the point at which the 4 pieces came together and most of the challenge was. The example project was built exactly as taken from the MERN example playlist without modification, a task which I was nearly about to skip.
Had to set up the vite dev proxy so /api reaches the backend, and understand why I was doing that-rather than just copying the config-, before getting to registration and login, with bcrypt hashing the password on the way in and on the way out, and signing a JWT and then the protect middleware reading the token from the Authorization: Bearer header, verifying it, and attaching req.user.
"Not authorized" with no further info because I was submitting the token in the body and not the header, then "jwt malformed" because I had written bearer${token} (missing space), so token.split(' ')[1] was undefined - about forty minutes for a single character. Then the server exiting as soon as I started running it because I was missing a JWT_SECRET value in .env

and jwt.sign threw "secretOrPrivateKey must have a value". And then an edit to .env that seemed like it hadn't had any effect because .env only loads once at server start up so I hadn't restarted.

The one lesson I'd keep if I could only keep one is the ownership check; I think it has to be kept. Every route that touches a document makes sure the document's user field is equal to req.user.id, else any logged-in user with a brain can delete the data belonging to anyone else by guessing an id. Authentication alone protects against nothing.

(I tested this thoroughly with a second browser profile and a second account.)

The other thing I decided on is to make the authentication failures 401s and the ownership failures 403s, which makes things simpler at the front end, as only a 401 logs the user out.

My login would always be lost on every refresh until I remembered to read the user out of localStorage on startup. Also, at one point login failed silently because I was hashing the password both in the controller and in a Mongoose pre-save hook.

The habit I have changed the most this week is what I do when I see a broken something. If I go to the browser the smallest amount of info I get is the DevTools Network tab where it says which side is broken and what the status code actually was, what I sent and what came back. If I look in the terminal running the server it gives me the actual error and a stack trace with the file and line that it came from.

If the browser gives me a 500 and there is nothing in the terminal then I know the server isn't running at all, if the browser gives me a 500 and there is a stack trace I know my code threw an error.

Simply having a 500 in the browser and nothing in the terminal is not helpful enough information to distinguish between those problems on my own.

This was also the week I most wished I had a commit history. When the login flow worked and then stopped working I had no pre-existing state to roll back to, so had to find the change by reading. I noted this and pressed on, which was the wrong call twice over.

I learned about,

- why the token goes in a header and not the body, and what the symptom looks like when it does not.
- the ownership check as the security lesson of this course, and the 401 versus 403 distinction.

- hashing with bcrypt in exactly one place.
- a debugging order: server terminal, then Network tab, then the browser.

25.7.-30.7.2026 - The Study Tracker, and finally putting it under version control

The course requires a project that differs from the example, and this is where I learned most, because nobody was narrating.

I decided the data on paper first - courses, sessions and goals, what fields each has, and which ones point at each other. Then I built one resource end to end (model, controller, routes, service, slice, page) for courses only, before starting the second. Sessions took roughly a quarter of the time, because the first one had become a template. Doing it the other way round - all the models, then all the controllers, then all the pages - would have meant finding out nothing worked until the very end, with everything broken at once. I also wrote a seed script early, after resetting the database once and clicking ten records back in by hand.

Two bugs are worth the space. First, deleting a course left its sessions behind pointing at nothing, and the session list then crashed on session.course.code because populate returned null. Deleting related documents is not optional.

Second, and this is the entry I would most want to be asked about: goal progress silently never worked. An <input type="date"> gives you midnight. A record created "now" carries the actual clock time. So session.date >= goal.createdAt was comparing midnight against something like 21:40 and was always false. Nothing threw and nothing logged - the number was simply always zero. Worse, it would have appeared to work if I had happened to test it early in the morning. A JavaScript Date is always a precise moment, never just a day; when I mean to compare days I now flatten both sides with setHours(0, 0, 0, 0).

For the stats endpoint I used find() and a loop rather than an aggregation pipeline. That is slower, and at one user's data size it is also something I can debug. I would rather be able to explain that decision than have reached for $lookup because it looks more advanced.

I then broke the app on purpose in three ways and fixed each: removed a required field from a model, removed express.json(), and removed an ownership check and then deleted another user's record from a second browser profile to confirm it really was possible. Reading my own error messages when I already know the cause is the cheapest debugging practice there is.

The last thing I did was put the project under version control, and doing it last is the mistake I would most want a future version of me to read about. git init, git add ., and the staging area filled with tens of thousands of files, because node_modules was still there and I had never written a .gitignore. I wrote one, ran git rm -r --cached node_modules, and committed again.

The part that actually mattered was worse. .env had gone in with MONGO_URI and JWT_SECRET in it. Deleting the file in a later commit does not remove it from the history - once a secret is committed it stays there - so I rotated the JWT secret and started the repository again from a clean tree with .gitignore in place before the first commit, rather than push a history with credentials in it. Pushing then failed on authentication, because GitHub wants a personal access token and not my account password, and while I pasted the token nothing appeared on screen at all; I assumed the terminal had frozen and cancelled twice before reading that hidden input is normal.

The course says to commit often, and I now understand why from the wrong side. My history is a handful of enormous commits made in one evening, so it is a record of when I remembered git existed rather than of how the project was built - and through the two weeks where things broke most, I had nothing small to go back to. I did at least break a file on purpose and recover it with git checkout -- ., which is the safety net I spent five weeks not having. Commit messages turn out to be the same idea: "fix" tells me nothing three weeks later, "Fix goal progress date comparison" tells me everything. On the next project the repository is the first thing, before the first npm install.

Where I am now: I can trace a single click from the React button through the axios call, the Express route, the middleware, the controller, Mongoose and MongoDB, and back to the re-

rendered component, and I can point at any file in the project and say what would break if I deleted it. The parts I could not explain in my own words, I rewrote from scratch until I could.

I learned about,

- building vertically through one feature instead of horizontally across layers.
- dates being moments rather than days, and how a comparison can fail silently and intermittently.
- cleaning up related documents on delete.
- choosing the implementation I can debug over the clever one, and being able to say why.
- why .gitignore has to exist before the first npm install, that a committed secret stays in the history, and that personal access tokens are not passwords.
- most of all, that version control is worth nothing until it is actually running - I learned this by doing it in the wrong order and paying for it.